

COMP90015: Distributed System Assignment 1- Multi-threaded Dictionary Server

Name: Karthik Kannan (1739619)

Course: Master of Computer Science (MC-CS)

Problem context of the assignment: The provided task to design and implement a multi-threaded dictionary server using client-server architecture. The model should allow multiple concurrent clients and perform the following operations of - searching for word meanings, add a new word, remove an existing word, add new meanings to an existing word and update an existing word.

The server must be able to handle concurrent clients safely ensuring that multiple clients can read the dictionary simultaneously while write operations are performed exclusively by one client at a time. The dictionary data must also be persisted to durable storage so that changes survive server restarts or crashes.

The model explicitly demonstrates two fundamental distributed systems concepts of sockets for communication and threads for concurrency at a low-level abstracted demonstration. The model consists of two separate applications – a server that hosts the dictionary and a client with a GUI that allows users to interact with the dictionary. Both are launched via CLI to avoid hardcoded configurations

A. Components of the system

The following components were used in the development of the model:

- i. `Dictionary.java`: is a shared data structure of the server. It stores the word and their respective meanings in the following format – `HashMap<String, List<String>>` which allows efficient word lookup. The dictionary data is loaded from a JSON file on server startup using `loadDictionary()` and saved back to the file after every write operation through `saveDictionary()`, ensuring persistence across server restarts. To handle concurrent access safely, a `ReentrantReadWriteLock` is used. The `search()` method acquires a read lock allowing multiple clients to search simultaneously, while a `addWord()`, `removeWord()`, `addMeaning()` and `updateMeaning()` acquire a write lock ensuring access during respective modification.
- ii. `ClientHandler.java`: acts like the bridge between client and the dictionary. When a client connects to the server, `DictionaryServerGUI` creates a new `ClientHandler` object and calls `handler.start()`, which triggers the `run()` method in a separate thread so each client gets its own thread running independently. Inside the `run()` method, the class sets up `BufferedReader` and `PrintWriter` to communicate with the client via the socket. It then continues to listen for incoming JSON request [through `while ((request = in.readLine()) != null)`], parses the `action` field to determine the operation requested, calls the appropriate method on the shared `Dictionary` object and sends back a JSON response. When `readLine()` returns null the loop ends and client disconnects.
- iii. `DictionaryServerGUI.java`: is a graphical dashboard for the server extending `JFrame`. It displays the server status, the number of active connections and server logs in real time. It also displays a start/stop button that allows manual control of the server. When its started, it creates a `Dictionary` object, calls

`loadDictionary()` method and launches a background thread that accepts incoming client connections in a loop. For each connection, it creates a `ClientHandler` thread and updates the active connection count accordingly. The `log()` method uses `SwingUtilities.invokeLater()` to safely update the GUI from background threads. It accepts two CLI arguments – the port number and the dictionary file path.

- iv. `DictionaryClientGUI.java`: is the graphical interface for the client (user facing), extending `JFrame` and built on top of the provided assignment skeleton file. It consists of the 5 functional operations, the server address, port number and sleep duration. It establishes a TCP socket connection to the server via `connectToServer()` and sets up `PrintWriter` and `BufferedReader` for communication. Each operation builds a JSON request, sends it to the server via `out.println()`, simulates a delay using `Thread.sleep(sleepDuration)` and displays the JSON response in the Results panel.
- v. `dictionary.json`: A json data file to hold all the words and their respective meanings in a dictionary datatype format. This file is loaded on server startup and updated after every write operation, serving as the sole storage for the dictionary.

B. Class design and interaction diagrams

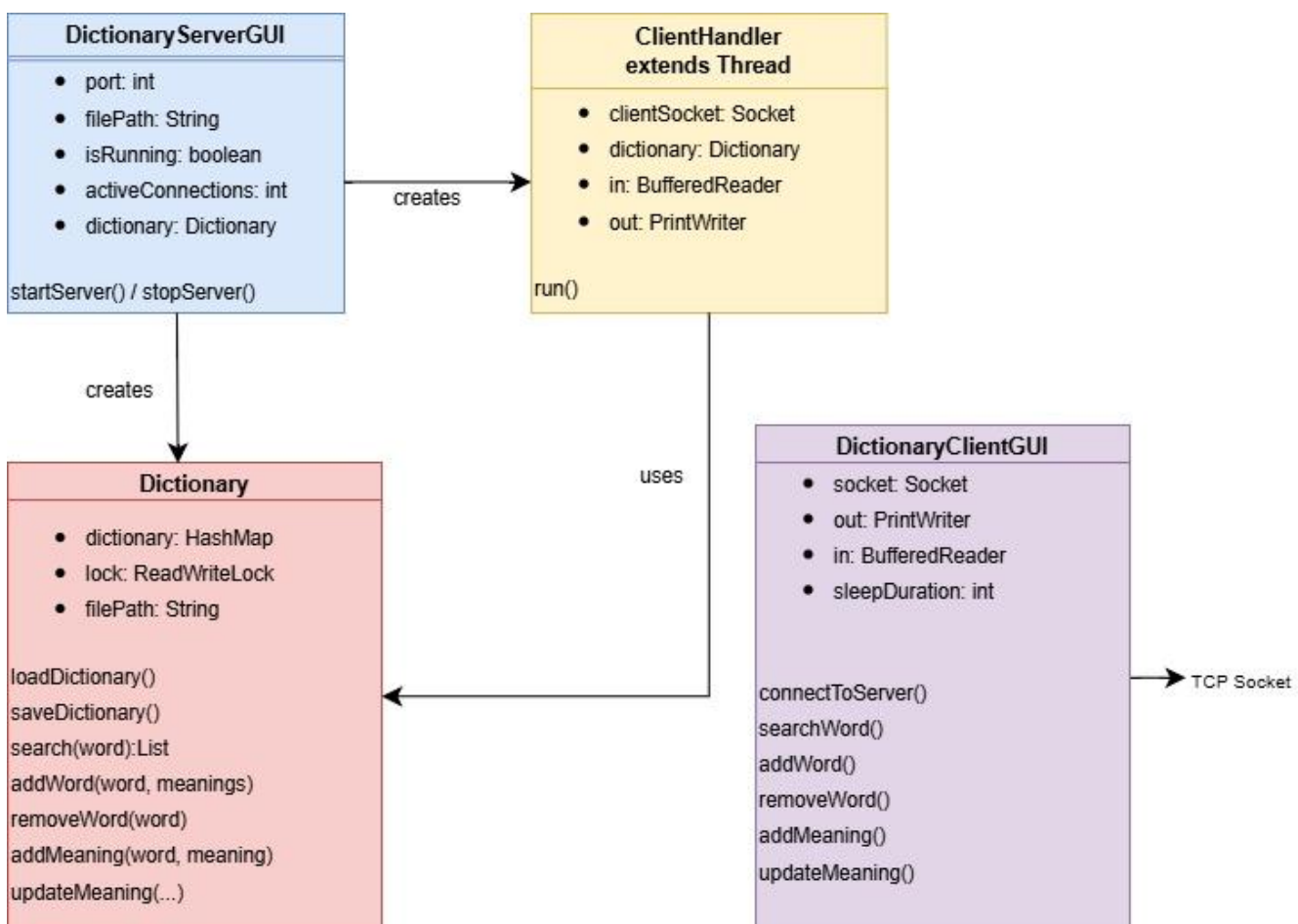


Figure 1. Class Design diagram

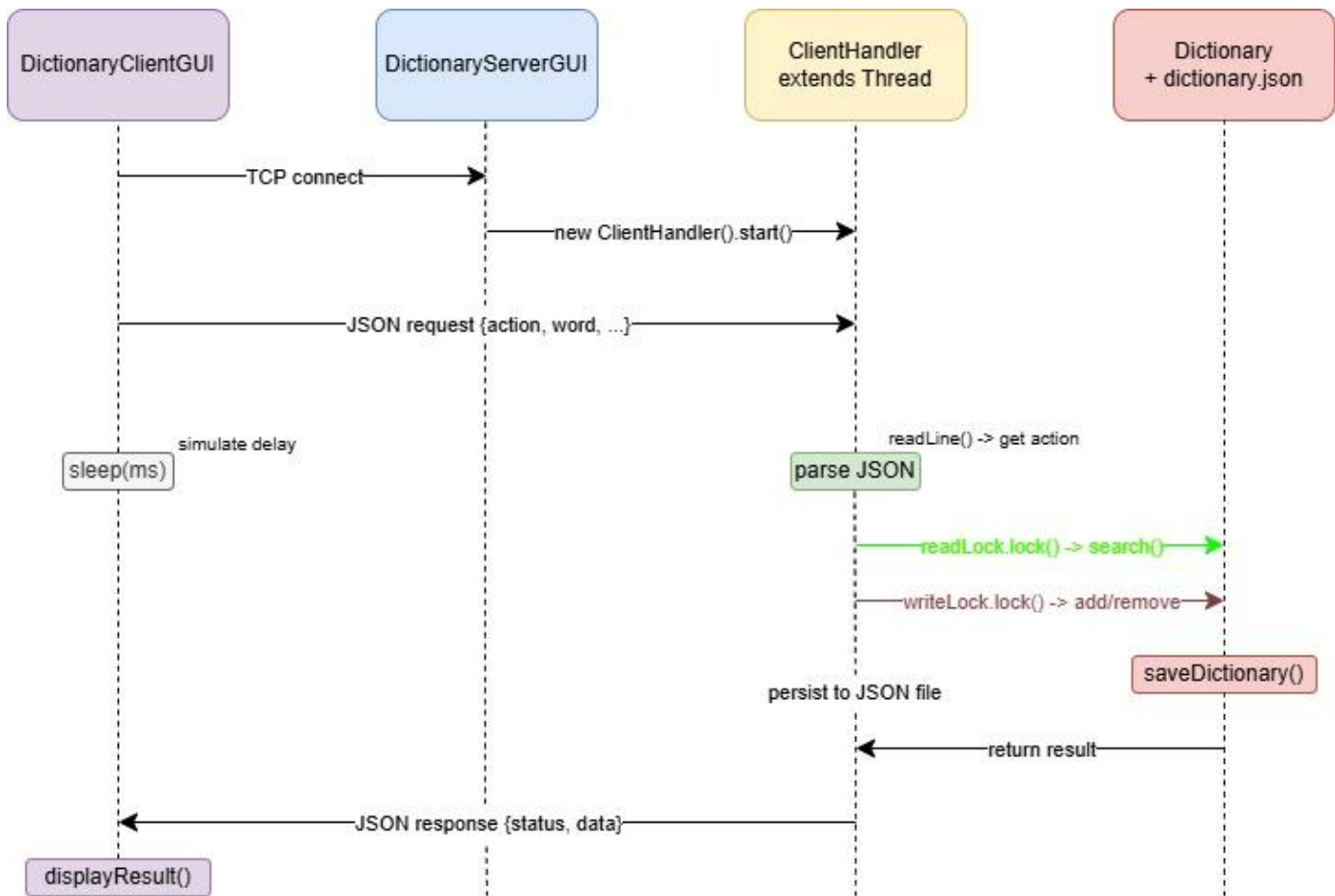


Figure 2. Interaction Diagram

Note: draw.io was used to create the above UML diagrams.

C. Critical analysis of the model

- Why TCP over UDP is chosen or vice-versa in your design? Explain

I used TCP over UDP because it provides guaranteed, ordered and reliable delivery of data between the client and server. There is a possibility of a corrupted or lost packet in case with UDP which could be detrimental to the dictionary. TCP also handles this automatically through its built-in acknowledgement and retransmission mechanisms, meaning no additional reliability layer is needed to be implemented. UDP while being faster would require a custom reliability layer which will also add more complexity for the use cases. Finally, TCP's persistent connection model suits the problem statement well since the socket remains open for the entire client-server session, allowing multiple operations without reconnecting.

- Why a specific messaging exchange protocol e.g., XML is used over others? Explain

I chose JSON as the messaging exchange protocol for the following reasons:

- Communication between client and server
- Persistent file storage

JSON fits well because the dictionary is basically a key-value pair where each word is a key paired with its respective list of meanings as value. Compared to XML, JSON is more compact and easier to parse. Java's built-in `org.json` library provides straightforward methods such as `JSONObject.put()`, `getString()` and `getJSONArray()` making ordering and unordering easy. JSON is also human-readable making debugging simpler, that is the 'dictionary.json' file can be opened in any text editor to verify its contents. Using the same format for communication and storage both also keeps the implementation consistent and reduces complexity reducing errors or inconsistencies in the functional use cases.

- How did you ensure or implement the “concurrent READS and restrict concurrent WRITES”? Explain

I implemented concurrent READS and restrict concurrent WRITES using `ReentrantReadWriteLock` (Tutorial 4) from `java.util.concurrent.locks`. This lock has two modes – a read lock and a write lock. The `search()` method acquires the read lock using `lock.readLock().lock()` allowing multiple threads to read simultaneously. The other 4 operations each leverage the write lock using `lock.writeLock().lock()`. The write lock allows only one thread to hold the write lock blocking all other threads from attempting to read or write until the functionality is completed and the lock is unlocked at the end. In all cases, the lock is eventually released in a finally block to guarantee it is always unlocked even if an exception occurs. This is better than using `synchronized` which would block all operations including reads.

- What are the pros/cons of the chosen architecture (client-server), thread-model (thread-per-object or thread-per connection or thread-pool) or thread-safety mechanism (allow concurrent READS and disable concurrent WRITES)?

Architecture: client-server

Pros	Cons
i. Clear separation of concerns between client and server ii. The dictionary is centralized so all clients always see the same data iii. Changes made by one client are immediately visible to other clients	i. It has a single point of failure ii. If the server goes down all clients will loose access

Model: thread-per-connection

Pros	Cons
i. Simple to implement ii. Each client gets a dedicated thread so operations don't queue behind each other iii. Isolation between clients like containers.	i. If a very large number of clients connect simultaneously it could exhaust system resources ii. Not scalable for high traffic scenarios.

Safety Mechanism: `ReentrantReadWriteLock`

Pros	Cons
i. Allows true concurrent reads which improves performance when multiple clients are searching at the same time ii. More fine-grained control than Synchronized	i. Complex to implement correctly than Synchronized ii. If write operations are more frequent then the read lock may rarely be used

- What is the purpose of introducing an intended delay or sleep argument in the server operations? Does this affect the design choices made? Explain how?

In my implementation I added a sleep argument (`<sleep-duration>`) to simulate real-world network latency. Its primary purpose is to expose and verify the thread safety of the server with a delay in place. If two clients perform write operations simultaneously, the delay holds the write lock for longer and making it easy to observe that concurrent writes are properly serialized. Without the delay, operations will execute instantly which it makes it near impossible to observe true concurrency. In real world, this way of artificially appending a delay to the model is not implemented. This does not affect the core design choices as the locking mechanism functions identically with or without the delay and also since it is purely a testing tool to verify that `ReadWriteLock` behaves correctly when threads genuinely compete for the same resource at the same time.